Enterprise AI Platform — Architecture & Engineering Handbook

Version: 1.0
Date: June 2026
Status: Published
Audience: Senior Engineers, Staff Engineers, Principal Engineers, Architects, AI Platform Engineers

---

Table of Contents

---

1. Executive Summary

1.1 What Is the Enterprise AI Platform?

The Enterprise AI Platform is a **reusable, modular monolith** implemented in Java 21 and Spring Boot 3.3. It provides the infrastructure for building AI-powered enterprise applications: retrieval-augmented generation (RAG), semantic enrichment, knowledge graph construction, multi-provider AI orchestration, evaluation, explainability, and production observability.

The platform is **not** a finished application. It is a **foundation** — a set of reusable modules, interfaces, and architectural patterns that support building AI-powered applications on top of a common core.

1.2 Why Does It Exist?

Most AI demonstrations are single-file Python scripts that call an LLM API. They lack modularity, testability, observability, and production resilience. The Enterprise AI Platform demonstrates how an experienced engineering team would integrate AI into enterprise software: with clear boundaries, abstract interfaces, graceful degradation, and full auditability.

The platform serves three purposes:

1. **Reference implementation**: Demonstrates production-grade AI engineering in Java
2. **Reusable foundation**: Supports multiple AI applications from a single codebase
3. **Portfolio demonstration**: Shows Staff Engineer-level architecture and engineering judgement

1.3 Scope

In scope:

- Multi-provider AI orchestration (Ollama, OpenAI-compatible)
- Retrieval-augmented generation (RAG) with hybrid search
- Semantic enrichment with automatic entity/concept extraction
- GraphRAG with Neo4j knowledge graph traversal
- Prompt registry with versioning and categories
- Model capability registry with intelligent routing
- Evaluation framework (grounding, faithfulness, hallucination detection)
- Full explainability metadata on every inference
- Workflow engine for multi-step document processing
- Production observability (Micrometer, Prometheus, health indicators)
- Graceful degradation for all external dependencies
- 157 automated tests across 5 testing layers

Out of scope:

- Microservices deployment (the platform is a modular monolith)
- Multi-agent AI systems
- Model fine-tuning infrastructure
- User management / multi-tenancy
- Billing / usage tracking
- Real-time collaboration

1.4 Target Use Cases

The platform supports building applications for:

- **Document Intelligence**: Upload, index, search, and analyze document collections
- **Contract Intelligence**: Extract obligations, parties, dates, and terms
- **Enterprise Search**: Hybrid search across organizational knowledge bases
- **Compliance**: Policy analysis, regulatory mapping, requirement extraction
- **Financial Analysis**: Report analysis, entity extraction, relationship mapping
- **Research Platforms**: Literature analysis, citation networks, concept discovery

The included **Document Intelligence** application serves as the first reference implementation. Future applications are built on the same platform core.

1.5 Non-Goals

The platform does **not** aim to be:

- A no-code AI platform (it is a developer framework)
- A SaaS product (it is self-hosted infrastructure)
- A chatbot framework (it is an enterprise AI platform)
- A model training/inference service (it orchestrates external providers)

Related ADRs: [ADR-001], [ADR-020]

---

2. Platform Vision

2.1 AI as Infrastructure

The central thesis of the platform is: **AI is infrastructure, not a feature**.

LLMs, embedding models, vector databases, and knowledge graphs are infrastructure components — analogous to databases, message queues, and caches. They should be abstracted behind interfaces, selected via configuration, and swapped without changing business logic.

This contrasts with the common pattern of embedding provider-specific code directly in application logic. In this platform, a `DocumentService` never calls `Ollama`. It calls `EnrichmentService`, which may be backed by Ollama, OpenAI, or a regex fallback.

2.2 Domain Independence

The platform core contains **no domain-specific logic**. There are no assumptions about legal documents, financial reports, or any specific industry. Domain customization happens through the `DomainConfiguration` SPI — a pluggable interface that provides concept definitions, analysis objectives, finding hierarchies, and system instructions.

The Document Intelligence reference application provides default configurations. New domains add their own `DomainConfiguration` implementations without modifying platform code.

Related ADR: [ADR-017]

2.3 Platform vs Application

A critical architectural distinction runs through the entire codebase:

- **Platform modules** (`platform-audit`, `platform-auth`, `platform-document`, `platform-search`, `platform-ai`, `platform-neo4j`, `platform-workspace`, `platform-observability`) provide infrastructure. They define SPIs, orchestration, and reusable services.
- **Application module** (`platform-api`) wires the platform together into a runnable application. It provides REST controllers, Thymeleaf templates, and application-specific configuration.

Future applications (Contract Intelligence, Financial Analysis, Compliance) would create their own application modules that depend on the same platform modules.

2.4 Reference Application Concept

The Document Intelligence application demonstrates how to use the platform. It includes:

- Document upload and ingestion (PDF, DOCX, TXT, HTML)
- Semantic enrichment during ingestion
- Hybrid search with keyword, vector, and graph retrieval
- AI-powered question answering with full grounding and evaluation
- Workspace management with 5-phase workflow wizard
- Multi-language UI (English, German, French)

- Audit logging with correlation IDs
- Health endpoints and Prometheus metrics

This application is both functional and educational — it shows how to wire the platform together.

Related ADR: [ADR-020]

---

3. Design Philosophy

3.1 Eight Governing Principles

The platform is governed by eight principles. Every architectural decision is evaluated against these. When a decision conflicts with a principle, it is either rejected or the principle is amended with explicit rationale.

#	Principle	ADR
1	**AI is Infrastructure** — LLMs, embeddings, and vector stores are abstracted behind interfaces	ADR-003
2	**Modular Monolith** — Compile-time module boundaries enforce dependency direction	ADR-001
3	**Graceful Degradation** — Every external dependency is optional; the platform starts with zero infrastructure	ADR-016
4	**Explainability by Default** — Every inference produces auditable metadata	ADR-012
5	**Documents Are the Knowledge Source** — Knowledge is extracted from documents during ingestion, never manually curated	ADR-007
6	**Domain Independence** — Platform core contains no domain-specific logic	ADR-017
7	**Testing as Architecture Documentation** — Tests validate behavior; test structure mirrors architecture	ADR-019
8	**Production Readiness** — Observability, health checks, and configuration are first-class concerns	ADR-015

3.2 Modularity

Module boundaries are enforced at **compile time** through Maven's dependency resolution. The dependency graph is directional — higher-level modules depend on lower-level APIs, never the reverse. `platform-api` is the assembly module that wires everything together; no other module depends on it.

```
platform-api
  ├── platform-ai
  │     ├── platform-search
  │     │     └── platform-document
  │     │           └── platform-audit
  │     ├── platform-neo4j
  │     └── platform-audit
  ├── platform-auth ── platform-audit
```

```
  ├── platform-workspace ── platform-document, platform-audit
  └── platform-observability
```

Each module has a single, well-defined responsibility. When a module grows too large (e.g., `platform-ai`), its internal package structure provides further separation without adding Maven modules.

3.3 Extensibility

Extension points are defined as **Service Provider Interfaces (SPIs)**. The platform defines the contract; implementations provide the behavior. Key SPIs:

SPI	Location	Purpose
`ChatCompletionProvider`	`platform-ai/api/`	LLM backends
`EmbeddingProvider`	`platform-search/api/`	Embedding generation
`GraphSearchProvider`	`platform-search/api/`	Graph-based retrieval
`VectorSearchProvider`	`platform-search/api/`	Vector similarity search
`EnrichmentService`	`platform-ai/api/`	Semantic enrichment
`EvaluationService`	`platform-ai/api/`	Answer evaluation
`DomainConfiguration`	`platform-ai/api/`	Domain-specific rules
`WorkflowEngine`	`platform-ai/api/`	Process orchestration

New implementations are activated by `@Component` + `@ConditionalOnProperty`. No existing code changes are needed.

3.4 Configuration Over Specialization

Provider selection, model routing, prompt choice, and retrieval strategy are all driven by **configuration**, not code. The `ProviderRouter` selects providers based on model name prefix and capability registry data. The `RetrievalOrchestrator` selects strategies based on classified intent. Conditional beans activate and deactivate based on property presence.

This means the platform can serve different domains with different providers simply by changing `application.yml` — no code changes, no rebuilds.

3.5 Testing Philosophy

Tests are **executable architecture documentation**. A Staff Engineer should understand how the platform works by reading the test suite. Tests describe behavior, not implementation:

```
✅ "routes openai:gpt-4o to openai provider"
✅ "skips unavailable provider and selects available one"

❌ "testProviderRouter"
❌ "testGetLatest"
```

The 157 tests span 5 layers: unit, integration, architecture, contract, and browser (Playwright). The test structure mirrors the platform architecture.

Related ADRs: [ADR-020], [ADR-019]

---

4. System Context

The Enterprise AI Platform is a single deployable Spring Boot application communicating with four external systems: PostgreSQL (required), Qdrant (optional vector DB), Neo4j (optional graph DB), and Ollama/OpenAI (optional LLM). All optional dependencies degrade gracefully.

[Architecture Diagram 01 — System Context: Show platform as central box with external actors (Browser, Ollama, OpenAI, Qdrant, Neo4j, PostgreSQL) connected via labeled arrows indicating protocol (HTTP REST, Bolt, JDBC).]

Technology Stack

Layer	Technology	Version
Language	Java	21
Framework	Spring Boot	3.3.5
Database	PostgreSQL (prod), H2 (test)	pgvector:pg16
Vector DB	Qdrant	latest
Graph DB	Neo4j	5-community
LLM	Ollama (local), OpenAI-compatible (cloud)	—
Metrics	Micrometer + Prometheus	via Boot BOM
UI	Thymeleaf + Bootstrap 5	via Boot BOM
Testing	JUnit 5, Spring Boot Test, Playwright	—

Related: ADR-001, ADR-002

---

5. Module Overview

platform-audit (leaf): Immutable audit log. JDBC implementation. Correlation IDs. All other modules depend on it.

platform-auth: JWT (HS256) + BCrypt (strength 12) + refresh token rotation. Form login + stateless API auth.

platform-document: Document lifecycle. PDFBox/POI/JSoup text extraction. Scheduled ingestion worker (10s poll).

platform-search: Hybrid search with keyword (JPA/TF), vector (Qdrant), graph (Neo4j) fusion. Weighted linear combination (k*0.40* + *v*0.40 + c*0.20). Sentence-aware chunking (1200-char target, 150-char overlap). Two-stage reranking.

platform-ai: Heart of the platform. 31 interfaces across RAG orchestration, enrichment, evaluation, prompt registry (8 prompts, 6 categories), model capability registry (6 models), provider router (4-tier selection), retrieval orchestrator, workflow engine, DomainConfiguration SPI.

platform-neo4j: Knowledge graph persistence. Auto-generated during ingestion. NodeProvenance on every element. Optional (conditional on platform.neo4j.uri).

platform-workspace: 5-phase wizard (SETUP->INGESTION->ANALYSIS->REVIEW->COMPLETE). Document linking, timeline events.

platform-observability: Micrometer + Prometheus. Health indicators (Ollama, Qdrant, providers). AiMetrics. Reusable across applications.

platform-api: Assembly module. REST controllers, Thymeleaf UI, SearchInfrastructureConfig for conditional bean wiring.

Related: ADR-001

---

6. Runtime Architecture

Startup: Component scan -> @ConfigurationProperties binding -> @ConditionalOnProperty evaluation -> Registry seeding -> Health indicator discovery -> Tomcat start.

DI: Constructor injection exclusively. Optional deps via ObjectProvider. No field injection.

Provider discovery: @Component beans gated by @ConditionalOnProperty. ProviderRouter receives List. Selection: model prefix -> capability -> preferred -> first available.

Related: ADR-002, ADR-003, ADR-016

---

7. Document Lifecycle

[Architecture Diagram 03 — Document Ingestion Pipeline: Sequence diagram showing Upload -> DocumentService -> IngestionWorker -> IngestionProcessor -> TextExtraction -> Enrichment -> Chunking -> Embedding -> PostgreSQL+Qdrant+Neo4j.]

Pipeline Steps

1. **Upload**: HTTP POST -> DocumentService.createDocument() creates Document + IngestionJob (PENDING)
2. **Scheduled Poll**: DocumentIngestionWorker polls every 10s for PENDING jobs
3. **Processing**: DefaultDocumentIngestionProcessor dispatches to IndexingOrchestrationService or keyword-only fallback
4. **Extraction**: TextExtractionService (PDFBox for PDF, POI for DOCX, JSoup for HTML)
5. **Enrichment**: EnrichmentHook extracts entities/concepts/relationships via EnrichmentService (LLM or regex), persists to Neo4j if available
6. **Chunking**: SentenceAwareChunkingStrategy splits at sentence boundaries (1200-char target, 150-char overlap)
7. **Embedding**: OllamaEmbeddingProvider generates 768-dim vectors via Ollama /api/embeddings
8. **Persistence**: Chunks to PostgreSQL (JPA), vectors to Qdrant (REST), graph to Neo4j (Bolt)

Graceful Degradation in Ingestion

- No embedding config -> keyword-only indexing (no vectors)
- No Qdrant -> vectors skipped, keyword search works
- No Neo4j -> enrichment runs, graph persistence skipped
- No LLM -> regex-based enrichment fallback

Related: ADR-007, ADR-010, ADR-016

---

8. AI Architecture

[Architecture Diagram 04 — AI Inference Pipeline: Sequence diagram showing User Query -> Intent Classification -> Retrieval Strategy -> Search (keyword+vector+graph) -> Fusion -> Reranking -> Prompt Assembly -> ProviderRouter -> LLM -> Grounding -> Validation -> Evaluation -> Explainability -> Structured Answer.]

Full AI Pipeline

1. **Intent Classification**: QueryIntentClassifier.classify()
2. **Strategy Selection**: RetrievalOrchestrator maps intent to SearchMode
3. **Retrieval**: RetrievalAugmentationService -> SearchFacade -> HybridRetrievalService (keyword+vector+graph fusion)
4. **Context Assembly**: ContextAssembler builds PromptContext
5. **Prompt Building**: PromptBuilder renders template via PromptRegistry
6. **Provider Selection**: ProviderRouter selects ChatCompletionProvider
7. **LLM Inference**: ChatCompletionProvider.complete() -> raw answer
8. **Validation**: TemporalConsistencyValidator + ClaimValidator
9. **Grounding**: GroundingService -> ConfidenceProfile
10. **Evaluation**: EvaluationService -> grounding, faithfulness, hallucination scores
11. **Explainability**: RetrievalOrchestrationResult captures full metadata
12. **Response**: AiResponse(ReasonedAnswer, InferenceMetadata)

Provider SPI

Interface	Implementations	Activation
ChatCompletionProvider	OllamaChatProvider, OpenAiChatProvider	@ConditionalOnProperty
EmbeddingProvider	OllamaEmbeddingProvider	@ConditionalOnProperty
RerankingProvider	OllamaRerankingProvider	@ConditionalOnProperty
VectorSearchProvider	QdrantVectorSearchProvider, NoOpVectorSearchProvider	Conditional on host
GraphSearchProvider	Neo4jGraphSearchAdapter, NoOpGraphSearchProvider	Conditional on Neo4j

Related: ADR-003, ADR-004, ADR-008, ADR-011

---

9. Semantic Enrichment

[Architecture Diagram 05 — Semantic Enrichment Engine: Data flow diagram showing Document Text -> Entity Extraction (ORGANIZATION, PERSON, TECHNOLOGY, REGULATION) -> Concept Extraction (temporal, financial, domain) -> Relationship Extraction (REFERENCES, PART_OF, RELATED_TO) -> Graph Persistence (Neo4j with NodeProvenance).]

The Semantic Enrichment Engine automatically extracts structured knowledge from documents during ingestion. Users never manually create knowledge entries. The document corpus IS the knowledge source.

Extraction Pipeline

1. **Entity Extraction**: Regex patterns for ORGANIZATION, PERSON, DATE, MONEY. LLM-based extraction via entity-extraction/v1 prompt when available.
2. **Concept Extraction**: Domain-specific concepts with confidence scores and related entities.

3. **Relationship Extraction**: Typed relationships (REFERENCES, PART_OF, RELATED_TO, MENTIONS) between entities and concepts.

EnrichmentHook

Bridges enrichment results to Neo4j: EnrichmentContext -> EnrichmentResult -> GraphNode/GraphRelationship -> Neo4j. Runs as part of DefaultDocumentIngestionProcessor. Gracefully degrades when enrichment or Neo4j is unavailable.

Provenance

Every node and relationship carries NodeProvenance: sourceDocumentId, chunkId, chunkOffset, extractionConfidence, extractionTimestamp, extractionModel, promptVersion, provider.

Related: ADR-007, ADR-018

---

10. GraphRAG

[Architecture Diagram 06 — GraphRAG Retrieval Flow: Three parallel retrieval streams (Keyword, Vector, Graph) converging into Fusion -> Reranking -> Candidates. Show graph stream as optional with dashed border. Show Neo4jGraphSearchAdapter bridging GraphEnrichmentService to GraphSearchProvider SPI.]

GraphRAG extends retrieval with knowledge graph traversal. When Neo4j is available, the retrieval orchestrator includes graph results alongside keyword and vector results in the fusion step.

Architecture

Neo4jGraphSearchAdapter bridges platform-neo4j (GraphEnrichmentService) to platform-search (GraphSearchProvider SPI). This adapter:

- Implements GraphSearchProvider interface
- Activated by @ConditionalOnBean(GraphEnrichmentService.class)
- Calls GraphEnrichmentService.traverse() for graph traversal
- Returns RetrievalCandidate objects for fusion

Fusion Integration

Three-way merge in DefaultHybridRetrievalService:

- Keyword results enter at full weight
- Vector results enter at full weight
- Graph results boost existing candidates via combineWithGraph() (+15% max)
- Graph-only candidates receive baseline scores

Graceful Degradation

When Neo4j is unavailable: GraphEnrichmentService bean not created -> Neo4jGraphSearchAdapter not created -> NoOpGraphSearchProvider active -> graph search returns empty -> keyword+vector continue normally.

Related: ADR-008, ADR-009, ADR-016

---

11. Retrieval Architecture

[Architecture Diagram 07 — Retrieval Orchestration: Decision tree showing Query -> IntentClassifier -> {INDEX_INSPECTION: Keyword, QUESTION_ANSWERING: Hybrid, DOCUMENT_LOOKUP: Semantic} -> Search

Execution -> Fusion -> Reranking -> Candidates.]

Intent Classification

QueryIntentClassifier.classify() maps natural language queries to intents using keyword rules. The classified intent drives strategy selection in RetrievalOrchestrator.

Strategy Selection (deterministic)

Intent	Strategy	Mode
INDEX_INSPECTION, CORPUS_DISCOVERY	KEYWORD_ONLY	KEYWORD
QUESTION_ANSWERING, WORKSPACE_ANALYSIS	HYBRID	HYBRID
DOCUMENT_RESEARCH, DOCUMENT_LOOKUP	SEMANTIC	SEMANTIC

Fusion Algorithm

Weighted linear combination: `score = (keyword*0.40 + vector*0.40 + confidence*0.20) * docTypeWeight + graphBoost`. Clamped to [0, 1]. Graph boost adds up to 15% for candidates with related graph nodes.

Reranking (two-stage)

1. DefaultRerankingService: sort by rankingScore descending
2. OllamaRerankingProvider (optional): LLM cross-encoder scores top 15 candidates, blends 80% LLM + 20% original score

Related: ADR-008, ADR-011

---

12. Prompt Architecture

Prompt Registry

Versioned prompts stored in DefaultPromptRegistry with 6 categories: RETRIEVAL, SUMMARIZATION, EXTRACTION, CLASSIFICATION, EVALUATION, REASONING, WORKFLOW, GRAPH, SEARCH, SYSTEM.

Prompt Lifecycle

1. Registered at startup in DefaultPromptRegistry constructor
2. Resolved by RetrievalOrchestrator via getLatest("rag-answer")
3. Rendered with variable substitution: {{context}}, {{question}}
4. Qualified ID recorded in RetrievalOrchestrationResult for audit

Categories

Category	Example Prompt	Use Case
RETRIEVAL	rag-answer/v1	RAG-grounded QA
EXTRACTION	entity-extraction/v1	Entity and concept extraction
EVALUATION	rerank-evaluation/v1	Cross-encoder relevance scoring
SYSTEM	intent-classification/v1	Query intent routing

GRAPH	graph-relation-extraction/v1	Knowledge graph population
SUMMARIZATION	document-summary/v1	Document summarization

Related: ADR-005

---

13. Model Architecture

Model Capability Registry

Seeded with 6 models (4 Ollama, 2 OpenAI) covering all capability dimensions: streaming, vision, JSON mode, tool calling, embeddings, reasoning, structured output, context window, output tokens, latency estimates, cost estimates, preferred use cases.

Provider Router (4-tier selection)

1. Model prefix: "openai:gpt-4o" -> openai provider
2. Capability match: request STREAMING -> provider with streaming-capable model
3. Preferred provider: explicitly requested provider
4. First available: first provider reporting isAvailable() == true

Adding a Model

```
registry.register(new ModelCapability("claude-3.5-sonnet", "anthropic",
    true, true, true, true, false, false, true,
    200000, 8192, 800, 3.0, 0.7,
    List.of("general", "analysis", "code"),
    List.of("cloud", "frontier")));
```

Adding a Provider

1. Implement ChatCompletionProvider
2. Add @Component + @ConditionalOnProperty
3. Provider appears automatically in ProviderRouter via List injection

Related: ADR-004, ADR-006

---

14. Workflow Engine

In-memory workflow engine with configurable step transitions. Two pre-registered workflows: document-intelligence (SETUP->INGESTION->ANALYSIS->REVIEW->COMPLETE, 5 steps) and batch-ingestion (INGEST->ENRICH->COMPLETE, 3 steps).

API: start(definitionId, context), advance(instanceId), previous(instanceId), findInstance(instanceId), listActive().

Steps declare handler types (manual, automated, ai) as extension points. Not yet enforced at runtime. Transitions are linear; conditional branching deferred to future.

Related: ADR-014

---

15. Explainability

Every inference produces RetrievalOrchestrationResult with: intent, strategy, provider, model, prompt template ID/version, timing (start/end), keyword/vector/graph result counts, fusion method, reranking status, trace log (step-by-step decisions), evaluation scores.

explain() method produces human-readable summary: "Intent: QUESTION_ANSWERING | Strategy: HYBRID | Provider: ollama | Model: qwen2.5:14b | Prompt: rag-answer/v1 | Sources: 15 | Fusion: weighted-linear-fusion | Reranking: yes | Duration: 245ms".

Related: ADR-012

---

16. Observability

Metrics (AiMetrics)

Metric	Type	Tags
ai.inference.duration	Timer (percentile histogram)	provider, model
ai.embedding.duration	Timer	provider, model
ai.retrieval.duration	Timer	mode
ai.graph.retrieval.duration	Timer	—
ai.enrichment.duration	Timer	—
ai.ingestion.duration	Timer	document_type
ai.provider.available	Gauge (0/1)	provider
ai.evaluation.*	Summary	metric name

Health Indicators

- OllamaHealthIndicator: GET / -> 200 = up
- QdrantHealthIndicator: GET /collections -> 200 = up
- ProviderHealthIndicator: aggregates ChatCompletionProvider.isAvailable() across all providers

Endpoints

- /actuator/health (with details)
- /actuator/prometheus
- /actuator/metrics
- /actuator/info

Related: ADR-015

---

17. Security

Authentication

JWT (HS256) via Nimbus JOSE + JWT library. BCrypt password hashing (strength 12). Refresh tokens SHA-256 hashed before storage. Token rotation on refresh. Configurable TTL: access 15min default, refresh 30d default.

Authorization

Role-based: USER, ANALYST, ADMIN. Stateless API auth (Authorization: Bearer header) + session-based form login (JSESSIONID). CSRF disabled (API-focused platform).

Provider Isolation

Provider API keys (OpenAI) stored in application.yml with env var substitution: ${OPENAI_API_KEY}. Never hardcoded. Ollama runs locally — no auth required.

Prompt Safety

Prompts are internal platform assets (not user-supplied). User input is injected as {{question}} variable into vetted prompt templates. HTML-escaping in controller responses prevents XSS.

Related: ADR-002

---

18. Testing Strategy

Summary of TESTING.md. 157 tests across 5 layers:

Layer	Count	Command	Purpose
Unit	44	mvn test	Isolated component behavior
Integration	91	mvn verify	Subsystem interaction with Spring context
Architecture	10	mvn verify	End-to-end behavioral validation
Contract	5	mvn verify	SPI stability protection
Playwright	12	mvn verify -Pui-tests	Browser user journeys

Test Corpus

test-corpus/ contains 3 documents (technical-spec, financial-report, contract-sample) with expected architectural behaviors. Tests validate enrichment, entity extraction, and concept detection against these documents.

CI Strategy

Default: mvn clean verify (unit + integration + architecture) UI profile: mvn verify -Pui-tests (Playwright, requires running app) Performance profile: mvn verify -Pperformance

Related: ADR-019, TESTING.md

---

19. Deployment

Requirements

- Java 21
- PostgreSQL (required, port 5433)
- Qdrant (optional, port 6333)
- Neo4j (optional, port 7687)
- Ollama (optional, port 11434)

Docker Compose

docker-compose.yml defines PostgreSQL + pgAdmin + Qdrant + Neo4j (graph profile). Start with: docker compose up -d (basic) or docker compose --profile graph up -d (with Neo4j).

Configuration

All configuration via application.yml with env var substitution: ${DB_URL:jdbc:postgresql://localhost:5433/docintel}. Platform prefix for custom config: platform.auth., *platform.ai.*, platform.search., *platform.neo4j.*.

Startup

mvn spring-boot:run -pl platform-api (requires docker compose up -d first)

---

20. Extending the Platform

Adding an LLM Provider

1. Implement ChatCompletionProvider (providerName, isAvailable, complete)
2. Add @Component + @ConditionalOnProperty
3. Configure in application.yml under platform.ai.{provider}.*
4. Register model capabilities in DefaultModelCapabilityRegistry
5. ProviderRouter automatically discovers via List injection
6. Add health indicator by implementing HealthIndicator

Adding a Prompt

```
promptRegistry.register(new PromptTemplate("my-prompt", 1,
    PromptTemplate.Category.RETRIEVAL, "My prompt description",
    "Template with {{variable}}", List.of("variable"),
    "text", List.of("*"), 0.3, List.of(), Map.of()));
```

Adding a Workflow

```
definitions.put("my-workflow", new WorkflowDefinition(
    "my-workflow", "My Workflow", "Description",
    List.of(
        new WorkflowStep("start", "Start", "Begin", "manual", Map.of(), List.of("next")),
        new WorkflowStep("next", "Next", "Continue", "automated", Map.of(), List.of())
    ), "start"));
```

Adding a Domain

Implement DomainConfiguration interface. Provide as @Component or @Bean. Domain-specific concept definitions, objectives, finding hierarchies, centrality weights, and system instructions are automatically available to the AI pipeline.

Adding a Retrieval Source

Implement GraphSearchProvider (or similar SPI). Add @Component. DefaultHybridRetrievalService discovers via constructor injection. Results participate in fusion automatically.

Related: ADR-003, ADR-005, ADR-014, ADR-017

---

21. Performance Considerations

Chunking

- 1200-char target with sentence-boundary awareness
- 150-char overlap ensures context continuity
- Configurable via ChunkingStrategy SPI

Embedding

- Sequential embedding in current implementation
- Batch embedding API exists (embedBatch) but loops sequentially
- 768-dimensional vectors from nomic-embed-text
- Parallel embedding via virtual threads would improve throughput

Retrieval

- Weighted fusion is O(n) where n = total candidates
- Graph traversal depth limited to 2 hops
- No caching layer — each retrieval re-executes

LLM Inference

- Timeout configurable via platform.ai.ollama.request-timeout (default 120s)
- No streaming (returns complete response)
- No token-level latency tracking

Ingestion

- Scheduled worker polls every 10s, processes top 10 PENDING jobs
- No bulk/batch ingestion optimization
- Documents processed sequentially

---

22. Architectural Trade-offs

Modular Monolith vs Microservices

Decision: Modular Monolith. **Rationale**: Compile-time boundaries sufficient for current scale. No distributed systems complexity. Future extraction possible. (ADR-001)

Neo4j vs RDF Triple Store

Decision: Neo4j labeled property graph. **Rationale**: Cypher is more intuitive than SPARQL for traversal queries. No ontology management overhead. (ADR-009)

Qdrant vs pgvector

Decision: Qdrant as primary, pgvector as secondary. **Rationale**: Qdrant is purpose-built for vector search. pgvector works but Qdrant provides better performance at scale and native quantization. (ADR-010)

Provider Router vs Hardcoded Providers

Decision: Provider Router with SPI. **Rationale**: Adding providers requires zero changes to orchestration code. Selection is deterministic and auditable. (ADR-004)

Prompt Registry vs Inline Prompts

Decision: Registry with versioning. **Rationale**: Prompts are platform assets that evolve. Versioning enables audit trails and regression testing. (ADR-005)

Spring Boot vs Quarkus

Decision: Spring Boot 3.3. **Rationale**: Larger ecosystem, more familiar to enterprise teams, richer integration options. Quarkus offers faster startup but smaller community. (ADR-002)

Playwright vs Selenium

Decision: Playwright for browser tests. **Rationale**: Modern API, better reliability, auto-waits, parallel execution. Selenium is the legacy standard but Playwright provides better DX.

Weighted Linear Fusion vs Reciprocal Rank Fusion

Decision: Weighted linear fusion. **Rationale**: Simpler to implement and explain. RRF requires rank position tracking which is not available from all providers. The fusion method is accurately named in documentation. (ADR-008)

---

23. Lessons Learned

From Domain-Specific to Domain-Independent

The platform began as a German tenancy law analysis tool. Phase 1 removed domain-specific assumptions: 55 BGB paragraph numbers, 25+ German keywords, legal-specific enums and classifications. The result is a reusable platform. Lesson: domain-specific code is easier to remove when interfaces are clean.

Spring AI Removal

Spring AI 1.0.0 was added as a dependency but never used. Zero imports from org.springframework.ai.*. The build audit removed it and 5 transitive JARs. Lesson: regularly audit dependencies — unused libraries accumulate silently.

AI Pipeline Was Dead Code

The full AI pipeline (AiService with 12 collaborating services) compiled and passed all tests — but was never injected. The AiPageController only showed retrieval results without generating LLM answers. Lesson: bean wiring tests are critical for Spring applications.

GraphRAG Evolution

GraphRAG started as a documentation concept ("Neo4j participates in retrieval") but was never implemented. Phase 3 bridged the gap with Neo4jGraphSearchAdapter and wired it into the fusion pipeline. Lesson: document what IS implemented, not what SHOULD be implemented.

Prompt Registry Value

Prompt versioning was initially deferred but proved essential during testing — it enabled deterministic behavior validation and made prompt changes auditable. Lesson: version your prompts from day one.

Testing as Documentation

The test suite evolved from 128 wiring-verification tests to 157 behavioral tests that serve as executable architecture documentation. A Staff Engineer reading the tests can understand the platform without reading the implementation.

Lesson: invest in test naming and structure.

---

24. Future Evolution

Implemented

- Modular monolith with 9 modules
- Multi-provider AI (Ollama, OpenAI-compatible)
- Prompt registry (8 prompts, 6 categories)
- Model capability registry (6 models)
- Semantic enrichment (entities, concepts, relationships)
- GraphRAG with Neo4j traversal
- Retrieval orchestration (intent-driven strategy)
- Evaluation (grounding, faithfulness, hallucination)
- Explainability metadata on every inference
- Workflow engine (2 pre-registered workflows)
- Production observability (Micrometer, health checks)
- Graceful degradation for all external dependencies
- DomainConfiguration SPI
- 157 automated tests across 5 layers

Partially Implemented

- DomainConfiguration: SPI exists but 8 services still embed configurations
- AiMetrics: Component exists but not wired into all services
- WorkflowEngine: Engine exists but no PhaseHandler implementations
- GraphRAG: Works but uses simple entity-name matching, not graph embeddings
- Playwright: Tests written but require running application

Future Ideas

- Dynamic model capability discovery from provider APIs
- Resilience4j integration (retry, circuit breaker)
- OpenTelemetry distributed tracing
- Multi-domain routing (select DomainConfiguration per query)
- Graph embedding models for semantic graph traversal
- Streaming LLM responses (SSE)
- Prompt A/B testing framework
- Automated evaluation regression testing
- Spring Modulith for runtime module verification
- GraalVM native image for reduced startup time

---

25. Glossary

Term	Definition
Capability	A feature an AI model supports (streaming, vision, JSON mode, embeddings)
Chunk	A segment of document text (~1200 chars) stored and indexed for retrieval
DomainConfiguration	SPI for providing domain-specific rules (concepts, objectives, instructions)
Embedding	A 768-dimensional vector representation of text generated by an embedding model

Enrichment	Automatic extraction of entities, concepts, and relationships from documents
Evaluation	Automated quality assessment of AI-generated answers
Explainability	Metadata describing how an AI inference was produced (provider, model, prompt, strategy)
Fusion	Combining retrieval results from multiple sources into a single ranked list
GraphRAG	Retrieval-augmented generation enhanced with knowledge graph traversal
Intent	The classified purpose of a user query (question answering, document lookup, etc.)
Modular Monolith	Single deployable with compile-time module boundaries
Prompt Registry	Versioned store of prompt templates with categories and metadata
Provenance	Metadata linking a graph element to its source document and extraction method
Provider	An AI backend (Ollama, OpenAI) that implements a platform SPI
Provider Router	Component that selects which provider to use for a given inference request
RAG	Retrieval-Augmented Generation — grounding LLM responses in retrieved documents
Registry	A store of known entities (prompts, models, capabilities) with query capabilities
Retrieval Strategy	The selection of which retrievers to use (keyword, vector, graph) for a query
SPI	Service Provider Interface — a pluggable contract for extending the platform
Workflow	A configurable multi-step process with defined state transitions

---

26. References

Internal

- Architecture Decision Records — 20 ADRs documenting every major decision
- TESTING.md — Testing philosophy, layers, and execution guide
- README.md — Project overview and quickstart

External Technologies

- Spring Boot 3.3 Reference
- Qdrant Documentation
- Neo4j Cypher Manual
- Ollama API
- OpenAI API Reference
- Micrometer Documentation
- Playwright Documentation

---

27. Diagram Registry

The following diagrams should be produced in the next documentation phase:

ID	Title	Type	Description
01	System Context	C4 Level 1	Platform as central box, external actors, protocols
02	Module Overview	C4 Level 2	9 modules with dependency arrows
03	Document Ingestion Pipeline	Sequence	Upload -> Extraction -> Enrichment -> Chunking -> Indexing
04	AI Inference Pipeline	Sequence	Query -> Intent -> Retrieval -> Prompt -> LLM -> Validation -> Response
05	Semantic Enrichment Engine	Data Flow	Document -> Entity/Concept/Relationship extraction -> Graph persistence
06	GraphRAG Retrieval Flow	Data Flow	Three parallel streams (Keyword, Vector, Graph) converging into Fusion
07	Retrieval Orchestration	Decision Tree	Intent -> Strategy selection -> Search execution
08	Provider SPI Architecture	Class Diagram	Interfaces, implementations, conditional activation
09	Prompt Registry Structure	Entity Diagram	PromptTemplate, Category, Example, Version relationships
10	Model Capability Registry	Entity Diagram	ModelCapability, CapabilityRequest, Provider relationships
11	Testing Layers	Pyramid	Unit -> Integration -> Architecture -> Contract -> Playwright
12	Deployment Architecture	Deployment	Docker containers, ports, volumes, optional services

Do not create these diagrams in this phase. They are placeholders for the next documentation phase.